

Resumen Teórico – Práctico 02

Reducciones e Indecibilidad

Computabilidad y Complejidad (Teoría de la Computación II)

FCEFQyN – Universidad Nacional de Río Cuarto

Primer Cuatrimestre de 2026

Síntesis de la teoría necesaria para resolver el Práctico 02. Apoyada en el Capítulo 5 del libro de M. Sipser, *Introduction to the Theory of Computation*, y en las láminas de la cátedra ([Reducibilidad.pdf](#)).

Índice

1. Introducción	2
2. Repaso mínimo	3
2.1. Reconocibilidad y decidibilidad	3
2.2. Lenguajes de referencia indecibles	3
3. La idea de reducción (versión informal)	3
3.1. Estrategia general	3
3.2. Diagrama mental	3
3.3. Ejemplo prototipo: $HALT_{TM}$	4
4. Reducciones clásicas desde A_{TM}	4
4.1. E_{TM} : lenguaje vacío	4
4.2. $REGULAR_{TM}$: $\mathcal{L}(M)$ es regular?	5
4.3. EQ_{TM} : dos MT reconocen el mismo lenguaje?	5
4.4. Tabla resumen	5
5. Teorema de Rice	5
5.1. Propiedades semánticas	6
5.2. Enunciado y demostración	6
5.3. Aplicaciones inmediatas	6
6. Reducciones via historias de ejecución	7
6.1. Definición	7
6.2. Autómatas linealmente acotados (LBAs)	7
6.3. E_{LBA} es indecible	8
7. Problema de Correspondencia de Post (PCP)	8
7.1. Definición del problema	8
7.2. Ejemplo	8
7.3. Indecibilidad de PCP	9

8. Funciones computables y reducibilidad many-one	9
8.1. Funciones computables	9
8.2. Reducibilidad many-one (mapping reducibility)	9
8.3. Propiedades clave	9
8.4. Transitividad de $\leq_m$	10
8.5. Comportamiento bajo complemento	10
8.6. $\leq_m$ y regularidad	10
9. Caracterizaciones útiles	11
9.1. Reconocibilidad y A_{TM}	11
9.2. Si A reconocible y $A \leq_m \bar{A}$, entonces A decidible	11
9.3. $\overline{A_{TM}}$ no se reduce a E_{TM}	11
10. Técnicas para resolver el Práctico 02	11
11. Resumen final	12

1. Introducción

El Práctico 02 trata sobre *reducciones* entre problemas, una herramienta central de la teoría de la computación para clasificar problemas según su dificultad. Esto se construye sobre lo que ya sabemos del Práctico 01: máquinas de Turing, configuraciones, lenguajes reconocibles vs decidibles, y la existencia de problemas indecidibles (épicamente, A_{TM}).

El problema motivador. En el Capítulo 4 de Sipser se probó que A_{TM} es indecidible mediante un argumento de *diagonalización*. Pero esa técnica es delicada y muy ad-hoc. La pregunta natural es: *una vez que tenemos un problema indecidible (como A_{TM}), cómo sabemos que otros problemas también lo son?* La respuesta del Capítulo 5 es: por **reducción**.

Intuición. La idea es: si pudiera resolver el problema B , podría usarlo como “oráculo” para resolver el problema A . Si sabemos que A es imposible, entonces B también lo es. Esta observación simple es la herramienta principal para descubrir la indecidibilidad de muchos problemas naturales.

Hoja de ruta. Este resumen cubre, en orden:

- el repaso mínimo del Práctico 01 (§2);
 - la idea informal de reducción y cómo se usa para probar indecidibilidad (§3);
 - ejemplos clásicos de reducciones desde A_{TM} a $HALT_{TM}$, E_{TM} , $REGULAR_{TM}$, EQ_{TM} (§4);
 - el Teorema de Rice y sus aplicaciones (§5);
 - la técnica de *historias de ejecución*, con aplicaciones a autómatas linealmente acotados (§6);
 - el Problema de Correspondencia de Post (§7);
 - la **noción formal** de función computable y de reducibilidad many-one $\leq_m$ (§8);
 - las propiedades clave de $\leq_m$ y su uso en demostraciones (§??);
 - las técnicas específicas para los ejercicios del Práctico (§10).
-

2. Repaso mínimo

2.1. Reconocibilidad y decidibilidad

Definición 2.1 (Sipser 3.5–3.6). Sea $L \subseteq \Sigma^*$.

- L es *Turing-reconocible* si existe una MT M tal que $\mathcal{L}(M) = L$.
- L es *decidible* (o Turing-decidible) si existe un decidor para L , es decir, una MT M con $\mathcal{L}(M) = L$ que halta en toda entrada (nunca cicla).

Recordamos las inclusiones clásicas:

$$\text{REG} \subsetneq \text{CFL} \subsetneq \text{DEC} \subsetneq \text{REC} \subsetneq \mathcal{P}(\Sigma^*).$$

Además, un lenguaje L es decidible si y sólo si L y $\bar{L}$ son ambos reconocibles (Sipser 4.22).

2.2. Lenguajes de referencia indecidibles

Asumimos como conocidos los siguientes resultados de Sipser, Capítulo 4:

Teorema 2.2 (Sipser 4.11). $A_{\text{TM}} = \{\langle M, w \rangle \mid M \text{ es una MT y } M \text{ acepta } w\}$ es indecidible.

A_{TM} sí es *reconocible*: la máquina universal U que simula M con entrada w y acepta cuando M acepta es un reconocedor para A_{TM} . Por lo tanto A_{TM} es **reconocible pero no decidible**.

Corolario 2.3. $\overline{A_{\text{TM}}}$ no es reconocible. (Si lo fuera, junto con A_{TM} ya reconocible, A_{TM} sería decidible por Sipser 4.22.)

3. La idea de reducción (versión informal)

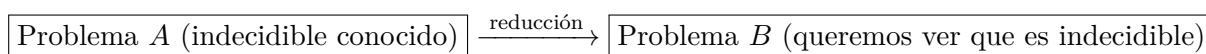
3.1. Estrategia general

Para mostrar que un problema B es indecidible (o no reconocible), la estrategia estándar es:

1. Tomar un problema A que ya sabemos que es indecidible (o no reconocible). Habitualmente $A = A_{\text{TM}}$ o $A = \overline{A_{\text{TM}}}$.
2. Mostrar que *si pudiéramos decidir B , podríamos decidir A* . En términos formales, mostrar que $A \leq_m B$ (o un vínculo equivalente).
3. Concluir: como A es indecidible, B también lo es.

Intuición. Pensar en A y B como dos problemas. Una reducción de A a B es una *traducción* algorítmica que convierte cualquier instancia de A en una instancia “equivalente” de B , en el sentido de que la respuesta es la misma. Si tuviéramos un decidor para B , lo aplicaríamos a la instancia traducida y obtendríamos la respuesta correcta para la instancia original de A .

3.2. Diagrama mental



Si tuviéramos un decidor de B , lo combinaríamos con la reducción para resolver A . Pero A no se puede resolver. Contradicción. Luego B no admite decidor.

3.3. Ejemplo prototipo: $HALT_{TM}$

Definición 3.1. $HALT_{TM} = \{\langle M, w \rangle \mid M \text{ se detiene con entrada } w\}$.

Teorema 3.2 (Sipser 5.1). $HALT_{TM}$ es indecidible.

Idea de la prueba. Reducimos A_{TM} a $HALT_{TM}$. Suponemos que R decide $HALT_{TM}$ y construimos un decidor S para A_{TM} :

$S =$ “Con entrada $\langle M, w \rangle$:

1. Correr R sobre $\langle M, w \rangle$.
2. Si R rechaza (M no se detiene), *rechazar*.
3. Si R acepta (M se detiene), simular M sobre w . Si M acepta, *aceptar*; si rechaza, *rechazar*.”

S siempre halta y decide A_{TM} , contradiciendo el Teorema 2.2. □

Esta es la plantilla de *todas* las pruebas de indecibilidad que veremos: asumir un decidor del nuevo problema, usarlo como subrutina para construir un decidor del problema viejo conocido como indecidible, y contradecir la hipótesis.

4. Reducciones clásicas desde A_{TM}

Esta sección resume las reducciones de Sipser 5.1–5.4, que son el *repertorio básico* para la mayoría de los ejercicios del Práctico 02.

4.1. E_{TM} : lenguaje vacío

Definición 4.1. $E_{TM} = \{\langle M \rangle \mid \mathcal{L}(M) = \emptyset\}$.

Teorema 4.2 (Sipser 5.2). E_{TM} es indecidible.

Idea. Reducción $A_{TM} \leq_m E_{TM}$. Dado $\langle M, w \rangle$, construimos M_1 :

$M_1 =$ “Con entrada x :

1. Si $x \neq w$, *rechazar*.
2. Si $x = w$, simular M sobre w y aceptar si M acepta.”

Observación clave: $\mathcal{L}(M_1) = \{w\}$ si M acepta w ; $\mathcal{L}(M_1) = \emptyset$ si no. Por lo tanto:

$$M \text{ acepta } w \iff \langle M_1 \rangle \notin E_{TM}.$$

Si E_{TM} fuera decidable, decidiríamos A_{TM} negando la respuesta: contradicción. □

4.2. $REGULAR_{TM}$: $\mathcal{L}(M)$ es regular?

Definición 4.3. $REGULAR_{TM} = \{\langle M \rangle \mid \mathcal{L}(M) \text{ es un lenguaje regular}\}$.

Teorema 4.4 (Sipser 5.3). $REGULAR_{TM}$ es indecidible.

Idea. Dado $\langle M, w \rangle$, construimos M_2 :

$M_2 =$ “Con entrada x :

1. Si x tiene la forma $0^n 1^n$ para algún $n \geq 0$, aceptar.
 2. En otro caso, simular M sobre w ; aceptar si M acepta.”
- Si M acepta w : $\mathcal{L}(M_2) = \Sigma^*$, regular.
 - Si M no acepta w : $\mathcal{L}(M_2) = \{0^n 1^n : n \geq 0\}$, no regular.

Por lo tanto M acepta $w \iff \langle M_2 \rangle \in REGULAR_{TM}$, y un decidor de $REGULAR_{TM}$ daría uno de A_{TM} . $\square$

4.3. EQ_{TM} : dos MT reconocen el mismo lenguaje?

Definición 4.5. $EQ_{TM} = \{\langle M_1, M_2 \rangle \mid \mathcal{L}(M_1) = \mathcal{L}(M_2)\}$.

Teorema 4.6 (Sipser 5.4). EQ_{TM} es indecidible.

Idea. Reducción $E_{TM} \leq_m EQ_{TM}$. Dado $\langle M \rangle$, sea $M_\emptyset$ una MT que rechaza toda cadena (luego $\mathcal{L}(M_\emptyset) = \emptyset$). Entonces:

$$\langle M \rangle \in E_{TM} \iff \mathcal{L}(M) = \mathcal{L}(M_\emptyset) \iff \langle M, M_\emptyset \rangle \in EQ_{TM}.$$

$\square$

4.4. Tabla resumen

Problema	Definición	Indecidible
A_{TM}	$\{\langle M, w \rangle : M \text{ acepta } w\}$	Sí (4.11)
$HALT_{TM}$	$\{\langle M, w \rangle : M \text{ se detiene con } w\}$	Sí (5.1)
E_{TM}	$\{\langle M \rangle : \mathcal{L}(M) = \emptyset\}$	Sí (5.2)
$REGULAR_{TM}$	$\{\langle M \rangle : \mathcal{L}(M) \text{ es regular}\}$	Sí (5.3)
EQ_{TM}	$\{\langle M_1, M_2 \rangle : \mathcal{L}(M_1) = \mathcal{L}(M_2)\}$	Sí (5.4)
All_{TM}	$\{\langle M \rangle : \mathcal{L}(M) = \Sigma^*\}$	Sí (Rice)

5. Teorema de Rice

El *Teorema de Rice* es una herramienta poderosa que generaliza muchas de las pruebas anteriores: cualquier propiedad “no trivial” que dependa únicamente del lenguaje reconocido por una MT es indecidible.

5.1. Propiedades semánticas

Definición 5.1 (Propiedad semántica). Un conjunto $P \subseteq \{\langle M \rangle \mid M \text{ es MT}\}$ se dice una *propiedad semántica* (o “de lenguajes”) si depende solo de $\mathcal{L}(M)$:

$$\mathcal{L}(M_1) = \mathcal{L}(M_2) \implies (\langle M_1 \rangle \in P \iff \langle M_2 \rangle \in P).$$

Intuición. Una propiedad semántica no distingue dos MTs que reconocen el mismo lenguaje, aunque tengan estructura interna distinta. Por ejemplo, “ $\mathcal{L}(M)$ contiene la cadena abc ” es semántica; “ M tiene exactamente 5 estados” *no* lo es, porque dos MTs distintas con distinta cantidad de estados pueden tener el mismo lenguaje.

Definición 5.2 (No trivialidad). P es *no trivial* si $P \neq \emptyset$ y $P \neq \{\langle M \rangle \mid M \text{ es MT}\}$. Es decir, hay alguna MT en P y alguna MT fuera de P .

5.2. Enunciado y demostración

Teorema 5.3 (Rice, Sipser Problema 5.28). *Toda propiedad semántica no trivial sobre MTs es indecidible.*

Idea de la prueba. Sea P una propiedad semántica no trivial. Sin pérdida de generalidad, asumimos que $\langle M_\emptyset \rangle \notin P$ (la MT que rechaza todo). En caso contrario, trabajamos con el complemento $\bar{P}$.

Como P es no trivial, existe $T \in P$, esto es, una MT testigo cuyo lenguaje está en P .

Reducimos A_{TM} a P . Dado $\langle M, w \rangle$, construimos M_w :

$M_w =$ “Con entrada x :

1. Simular M sobre w .
2. Si M acepta, simular T sobre x ; aceptar sii T acepta. (Si M no acepta, M_w tampoco; si M entra en bucle, M_w también.)”

Observación clave:

- Si M acepta w : $\mathcal{L}(M_w) = \mathcal{L}(T)$, luego $\langle M_w \rangle \in P$.
- Si M no acepta w : $\mathcal{L}(M_w) = \emptyset = \mathcal{L}(M_\emptyset)$, luego $\langle M_w \rangle \notin P$.

Así M acepta $w \iff \langle M_w \rangle \in P$, dando $A_{\text{TM}} \leq_m P$. Si P fuera decidable, A_{TM} también. Contradicción. $\square$

5.3. Aplicaciones inmediatas

Por Rice se obtiene de un golpe la indecidibilidad de:

- $E_{\text{TM}} = \{\langle M \rangle : \mathcal{L}(M) = \emptyset\}$.
- $All_{\text{TM}} = \{\langle M \rangle : \mathcal{L}(M) = \Sigma^*\}$.
- $\{\langle M \rangle : \mathcal{L}(M) \text{ contiene una cadena dada}\}$.
- $\{\langle M \rangle : \mathcal{L}(M) \text{ es regular}\}$, $\{\langle M \rangle : \mathcal{L}(M) \text{ es libre de contexto}\}$.
- $\{\langle M \rangle : \mathcal{L}(M) \text{ es finito}\}$, $\{\langle M \rangle : \mathcal{L}(M) \text{ es infinito}\}$.
- $\{\langle M \rangle : \mathcal{L}(M) \text{ es cerrado bajo reverso}\}$ (Ejercicio 3 del Práctico).

Para cada una basta verificar que la propiedad es semántica y no trivial.

Observacion. Rice *no* se aplica a propiedades *sintácticas* (referentes a la estructura de M , no a $\mathcal{L}(M)$). Por ejemplo, “ M tiene un estado inalcanzable” (Ejercicio 4 del práctico) es sintáctica: no es invariante bajo equivalencia de lenguajes. Para esa hace falta una reducción directa desde A_{TM} .

6. Reducciones via historias de ejecución

Algunos problemas de indecidibilidad se prueban más fácilmente codificando las *computaciones* de una MT como cadenas. Esto es la técnica de *computation histories*.

6.1. Definición

Definición 6.1 (Historia de ejecución, Sipser 5.5). Sea M una MT y w una entrada. Una *historia de ejecución aceptante* para M con w es una sucesión de configuraciones

$$C_1, C_2, \dots, C_l,$$

tal que:

- C_1 es la configuración inicial de M con entrada w ;
- cada C_{i+1} se obtiene de C_i aplicando una transición legal de M ;
- C_l es una configuración de aceptación (contiene q_{accept}).

Análogamente se define una historia *rechazante*.

Las historias son *finitas*. Las MTs deterministas tienen a lo sumo una historia para cada entrada. Si M entra en bucle sobre w , no existe historia (ni aceptante ni rechazante).

6.2. Autómatas linealmente acotados (LBAs)

Definición 6.2 (LBA, Sipser 5.6). Un *autómata linealmente acotado* (LBA) es una MT cuyo cabezal nunca se mueve fuera del segmento de cinta que ocupó originalmente la entrada. Si la transición quisiera moverlo más allá de un extremo, el cabezal se queda en su lugar.

Lema 6.3 (Sipser 5.8). Sea A una LBA con q estados y g símbolos en el alfabeto de cinta. Para una entrada de longitud n , A tiene a lo sumo

$$q \cdot n \cdot g^n$$

configuraciones distintas.

Intuición. Una configuración de un LBA está determinada por: el estado (q opciones), la posición del cabezal (n opciones, ya que la cinta tiene exactamente n celdas), y el contenido de la cinta (g^n posibilidades).

Teorema 6.4 (A_{LBA} es decidible, Sipser 5.9). $A_{LBA} = \{\langle A, w \rangle : A \text{ LBA y } A \text{ acepta } w\}$ es decidible.

Idea. Para decidir si A acepta w , simular A por hasta $q \cdot n \cdot g^n$ pasos. Si en ese rango A acepta, aceptar; si rechaza, rechazar; si no haltó, debe estar en bucle (por Lema 6.3: tiene que repetirse alguna configuración), así que rechazamos. $\square$

6.3. E_{LBA} es indecidible

Teorema 6.5 (Sipser 5.10). $E_{\text{LBA}} = \{\langle A \rangle : A \text{ LBA y } \mathcal{L}(A) = \emptyset\}$ es indecidible.

Idea (via historias). Reducimos $A_{\text{TM}} \leq_m \overline{E_{\text{LBA}}}$. Dado $\langle M, w \rangle$, construimos un LBA B que lee como input una pretendida historia de ejecución de M sobre w y verifica que sea una historia aceptante:

1. parsea la entrada como $\#C_1\#C_2\#\dots\#C_l\#$;
2. chequea que C_1 es la configuración inicial q_0w ;
3. chequea que cada C_{i+1} sigue legalmente de C_i según δ_M (un chequeo local entre cada par);
4. chequea que C_l contiene q_{accept} .

B realiza estos chequeos en espacio lineal sobre la entrada (que ya contiene la historia), así que es un LBA. Además:

$$\mathcal{L}(B) \neq \emptyset \iff M \text{ acepta } w.$$

Por lo tanto E_{LBA} resolvería A_{TM} (via su complemento), contradicción. □

7. Problema de Correspondencia de Post (PCP)

El PCP es un problema combinatorio aparentemente inocente que resulta ser indecidible.

7.1. Definición del problema

Una instancia del PCP es una colección finita de fichas (*dominos*):

$$P = \left\{ \left[\begin{smallmatrix} t_1 \\ b_1 \end{smallmatrix} \right], \left[\begin{smallmatrix} t_2 \\ b_2 \end{smallmatrix} \right], \dots, \left[\begin{smallmatrix} t_k \\ b_k \end{smallmatrix} \right] \right\},$$

donde cada $t_i, b_i \in \Sigma^*$ es una cadena no vacía.

Definición 7.1 (Matching). Un *matching* para P es una secuencia de índices $i_1, i_2, \dots, i_l$ (con repeticiones permitidas, $l \geq 1$) tal que

$$t_{i_1}t_{i_2}\dots t_{i_l} = b_{i_1}b_{i_2}\dots b_{i_l}.$$

$$\text{PCP} = \{\langle P \rangle : P \text{ es una instancia del PCP que tiene matching}\}.$$

7.2. Ejemplo

Sea $P = \left\{ \left[\begin{smallmatrix} ab \\ abab \end{smallmatrix} \right], \left[\begin{smallmatrix} b \\ a \end{smallmatrix} \right], \left[\begin{smallmatrix} aba \\ b \end{smallmatrix} \right], \left[\begin{smallmatrix} aa \\ a \end{smallmatrix} \right] \right\}$. La secuencia $(4, 4, 2, 1)$ es un matching:

$$\text{Arriba: } aa \cdot aa \cdot b \cdot ab = aaaabab,$$

$$\text{Abajo: } a \cdot a \cdot a \cdot abab = aaaabab.$$

Coinciden, así que $(4, 4, 2, 1)$ es un matching válido. (Este es el Ejercicio 5 del práctico.)

7.3. Indecibilidad de PCP

Teorema 7.2 (Sipser 5.15). *PCP es indecidible.*

Esquema. La prueba reduce $A_{TM} \leq_m MPCP \leq_m PCP$, donde MPCP (PCP modificado) exige además que el matching empiece con la primera ficha. La idea es: dado $\langle M, w \rangle$, construir una instancia P del MPCP cuyas fichas codifican las reglas de transición de M . Cada ficha permite “avanzar” un paso en una historia de ejecución de M sobre w . La instancia tiene matching si y solo si M acepta w .

Las fichas para una transición $\delta(q_i, a) = (q_j, b, R)$ son $\begin{bmatrix} q_i a \\ b q_j \end{bmatrix}$, y para $\delta(q_i, a) = (q_j, b, L)$ son $\begin{bmatrix} c q_i a \\ q_j c b \end{bmatrix}$ para todo $c \in \Gamma$. También hay fichas de copia $\begin{bmatrix} a \\ a \end{bmatrix}$ para cada símbolo. La construcción completa incluye fichas iniciales (que fuerzan el comienzo del matching), fichas de “limpieza” una vez alcanzado q_{accept} , y fichas de cierre. $\square$

8. Funciones computables y reducibilidad many-one

Hasta aquí hemos usado la palabra “reducción” de manera informal. La Sección 5.3 de Sipser introduce la **noción formal** que vamos a adoptar.

8.1. Funciones computables

Definición 8.1 (Función computable, Sipser 5.17). Una función $f : \Sigma^* \rightarrow \Sigma^*$ es *computable* si existe una máquina de Turing M_f tal que, comenzando con w en la cinta, M_f se detiene con $f(w)$ en la cinta.

Intuición. Las funciones computables son las que se pueden calcular mediante un algoritmo: dada la entrada, se obtiene la salida en tiempo finito. Las operaciones aritméticas, las transformaciones de cadenas, las modificaciones de descripciones de MTs, etc., son todas computables.

Ejemplo 8.2. La función que toma $\langle M, w \rangle$ y devuelve $\langle M_1 \rangle$, donde M_1 es la MT que rechaza toda entrada distinta de w y simula M sobre w cuando recibe w , es computable: basta extender mecánicamente la descripción de M con estados que comparen la entrada con w .

8.2. Reducibilidad many-one (mapping reducibility)

Definición 8.3 ($\leq_m$, Sipser 5.20). Sean $A, B \subseteq \Sigma^*$. Decimos que A es *reducible many-one a* B , escrito $A \leq_m B$, si existe una función computable $f : \Sigma^* \rightarrow \Sigma^*$ tal que, para toda cadena w :

$$w \in A \iff f(w) \in B.$$

La función f se llama una *reducción de A a B* .

Intuición. f traduce instancias de A a instancias de B preservando la respuesta. Si tuviéramos un decidor de B , lo aplicaríamos a $f(w)$ y obtendríamos la respuesta correcta para $w \in A$.

8.3. Propiedades clave

Las dos propiedades fundamentales son las siguientes:

Teorema 8.4 (Sipser 5.22). *Si $A \leq_m B$ y B es decidable, entonces A es decidable.*

Demostración. Sea M_B un decidor de B y M_f una MT que computa la reducción f . Definimos $N =$ “Con entrada w :

1. Computar $f(w)$ usando M_f .
2. Correr M_B con entrada $f(w)$ y devolver lo mismo.”

N siempre halta y N acepta w ssi M_B acepta $f(w)$ ssi $f(w) \in B$ ssi $w \in A$. Por lo tanto N decide A . $\square$

Corolario 8.5 (Sipser 5.23). *Si $A \leq_m B$ y A es indecidible, entonces B es indecidible.*

Este es el contrareciproco del teorema anterior y es la herramienta de trabajo: para probar que B es indecidible, exhibimos una reducción $A \leq_m B$ con A ya conocido como indecidible.

Teorema 8.6 (Sipser 5.28). *Si $A \leq_m B$ y B es reconocible, entonces A es reconocible.*

La prueba es idéntica a la del Teorema 8.4, salvo que M_B ahora es solo un reconocedor (puede ciclar en cadenas que no acepta).

Corolario 8.7 (Sipser 5.29). *Si $A \leq_m B$ y A no es reconocible, entonces B no es reconocible.*

8.4. Transitividad de $\leq_m$

Proposición 8.8 (Sipser, Ejercicio 5.6). *Si $A \leq_m B$ y $B \leq_m C$, entonces $A \leq_m C$.*

Demostración. Sean $f : \Sigma^* \rightarrow \Sigma^*$ y $g : \Sigma^* \rightarrow \Sigma^*$ las reducciones correspondientes. Definimos $h = g \circ f$. La composición de funciones computables es computable (basta concatenar las MT que las computan). Además,

$$w \in A \iff f(w) \in B \iff g(f(w)) \in C \iff h(w) \in C.$$

$\square$

8.5. Comportamiento bajo complemento

Lema 8.9 (Sipser, Ejercicio 5.7). *$A \leq_m B$ si y solo si $\bar{A} \leq_m \bar{B}$.*

Demostración. Sea f la reducción. Para todo w , $w \in A \iff f(w) \in B$, lo que equivale a $w \notin A \iff f(w) \notin B$, es decir $w \in \bar{A} \iff f(w) \in \bar{B}$. La misma f funciona en ambas direcciones. $\square$

Corolario 8.10. *Si $A \leq_m \bar{A}$ con la reducción f , entonces $\bar{A} \leq_m A$ con la misma f .*

8.6. $\leq_m$ y regularidad

Observación. La relación $\leq_m$ no preserva regularidad: si $A \leq_m B$ y B es regular, no se sigue que A sea regular. Contraejemplo: $A = \{0^n 1^n\}$ y $B = \{1\}$. Como A es decidable, podemos definir

$$f(x) = \begin{cases} 1 & x \in A, \\ 0 & x \notin A, \end{cases}$$

que es computable y reduce A a B . Sin embargo A no es regular. La moraleja es que $\leq_m$ es una relación de *computabilidad*, no de regularidad: usa MTs, no autómatas finitos.

9. Caracterizaciones útiles

9.1. Reconocibilidad y A_{TM}

Teorema 9.1 (Sipser, Ejercicio 5.22). *Para todo $A \subseteq \Sigma^*$:*

$$A \text{ es reconocible} \iff A \leq_m A_{TM}.$$

Demostración. ($\Rightarrow$) Sea M_A un reconocedor de A . Definimos $f(x) = \langle M_A, x \rangle$. La función es computable (la descripción de M_A está cableada). Se cumple $x \in A \iff M_A \text{ acepta } x \iff \langle M_A, x \rangle \in A_{TM}$.

($\Leftarrow$) Sigue del Teorema 8.6 y del hecho de que A_{TM} es reconocible. $\square$

Intuición. A_{TM} es “el problema reconocible por excelencia”: es *completo* para la clase de lenguajes Turing-reconocibles bajo $\leq_m$. Cualquier lenguaje reconocible se reduce a él.

9.2. Si A reconocible y $A \leq_m \bar{A}$, entonces A decidable

Proposición 9.2 (Sipser, Ejercicio 5.7). *Si A es reconocible y $A \leq_m \bar{A}$, entonces A es decidable.*

Demostración. Por el Lema 8.9, $A \leq_m \bar{A}$ implica $\bar{A} \leq_m A$ (con la misma función f). Como A es reconocible, por el Teorema 8.6, $\bar{A}$ también es reconocible. Entonces A y $\bar{A}$ son ambos reconocibles, y por Sipser 4.22, A es decidable. $\square$

Este es el Ejercicio 8 del práctico.

9.3. $\overline{A_{TM}}$ no se reduce a E_{TM}

Teorema 9.3 (Sipser, Ejercicio 5.5). $\overline{A_{TM}} \not\leq_m E_{TM}$. *Equivalentemente, no existe una función computable que reduzca A_{TM} a $\overline{E_{TM}}$.*

Idea. Si existiera tal reducción $A \leq_m \overline{E_{TM}}$, junto con A_{TM} reconocible se obtendría $\overline{E_{TM}}$ reconocible. Pero $\overline{E_{TM}}$ *sí* es reconocible (existe una MT que enumera todas las cadenas y, por dovetailing, acepta $\langle M \rangle$ cuando alguna w es aceptada por M). La sutileza está en otro lado: lo que se prueba es que $A_{TM} \leq_m E_{TM}$ *no* existe (no se confundir el sentido de la reducción). $\square$

10. Técnicas para resolver el Práctico 02

A continuación resumimos, por tipo de ejercicio, las técnicas que se aplican.

T1. Reducción directa para indecidibilidad (Ejercicios 2, 3, 4)

Plantilla. Para mostrar que un lenguaje B es indecidible:

1. Elegir A indecidible (típicamente A_{TM} o $\overline{A_{TM}}$).
2. Dado $\langle M, w \rangle$, construir un objeto $\langle M_w \rangle$ (otra MT, otra cadena) que codifica la pregunta “ M acepta w ?” como una instancia de B .
3. Verificar: $\langle M, w \rangle \in A \iff \langle M_w \rangle \in B$.
4. Concluir $A \leq_m B$ y aplicar el Corolario 8.5.

Variantes.

- Si la propiedad de B es *semántica* (depende solo de $\mathcal{L}(M)$, como en el Ejercicio 3 sobre cierre bajo reverso), se puede aplicar Rice (Teorema 5.3) para obtener una prueba más breve.
- Si la propiedad es *sintáctica* (depende de la estructura de M , como en el Ejercicio 4 sobre estados inalcanzables), Rice no aplica y hay que dar una reducción explícita.
- En propiedades del comportamiento (como “intentar mover el cabezal a la izquierda en el extremo” del Ejercicio 2), conviene construir M' que “ejecute M con w ” en su rutina, gatillando el comportamiento específico solo cuando M acepta.

T2. Encontrar un matching de Post (Ejercicio 5)

Para resolver una instancia específica del PCP no hay algoritmo general (es indecidible), pero para instancias chicas se puede:

- analizar las fichas y notar cuáles aportan “positivo” o “negativo” a la longitud de cada lado;
- identificar fichas de cierre (donde arriba y abajo coinciden);
- probar combinaciones cortas.

Para la instancia del práctico, $(4, 4, 2, 1)$ funciona y produce $aaaabab$ en ambos lados.

T3. Demostraciones generales sobre $\leq_m$ (Ejercicios 6, 7, 8, 9)

- **Transitividad (Ej. 6):** usar la composición $h = g \circ f$ y argumentar computabilidad por concatenación de MTs.
- **Regularidad bajo $\leq_m$ (Ej. 7):** dar contraejemplo ($A = \{0^n 1^n\}$, $B = \{1\}$).
- **Reconocible + $A \leq_m \bar{A} \Rightarrow$ decidible (Ej. 8):** aplicar el Lema 8.9 y el Teorema 4.22.
- **A reconocible sii $A \leq_m A_{\text{TM}}$ (Ej. 9):** Probar las dos direcciones; la directa usa $f(x) = \langle M_A, x \rangle$.

T4. Aplicación de Rice (Ejercicio 10)

Para mostrar la indecidibilidad de $All_{\text{TM}} = \{\langle M \rangle : \mathcal{L}(M) = \Sigma^*\}$ por Rice:

1. Verificar que la propiedad “ $\mathcal{L}(M) = \Sigma^*$ ” es semántica: depende solo de $\mathcal{L}(M)$.
2. Verificar no trivialidad:
 - testigo en All_{TM} : una MT que acepta toda cadena;
 - testigo fuera: una MT que rechaza toda cadena.
3. Concluir por Teorema 5.3.

11. Resumen final

Recapitulamos los conceptos centrales para el Práctico 02:

- La **idea de reducción**: resolver A usando un decidor de B . Permite probar indecidibilidad por contradicción.
- Las **reducciones clásicas** de Sipser 5.1–5.4: $A_{\text{TM}} \leq_m HALT_{\text{TM}}$, $A_{\text{TM}} \leq_m E_{\text{TM}}$, $A_{\text{TM}} \leq_m REGULAR_{\text{TM}}$, $E_{\text{TM}} \leq_m EQ_{\text{TM}}$.

- El **Teorema de Rice**: toda propiedad semántica no trivial sobre MTs es indecidible. Aplica automáticamente a muchos problemas naturales.
- Las **historias de ejecución**, técnica que codifica cómputos como cadenas; aplicada a E_{LBA} y al PCP.
- Los **autómatas linealmente acotados**: A_{LBA} es decidable pero E_{LBA} no.
- El **Problema de Correspondencia de Post**, indecidible, con una reducción indirecta vía historias.
- La **noción formal de función computable** y la **reducibilidad many-one** $\leq_m$: $w \in A \iff f(w) \in B$ para alguna f computable.
- Las **propiedades clave** de $\leq_m$: preserva decidibilidad (5.22) y reconocibilidad (5.28); es transitiva; se comporta bien bajo complemento; *no* preserva regularidad.
- La **caracterización**: A reconocible $\iff A \leq_m A_{TM}$.
- Las **técnicas** para los ejercicios.

Hilo conductor. El Práctico 02 cierra el círculo con el Práctico 01. Allí descubrimos qué cosas se pueden computar y por qué existen problemas que no son decidibles. Aquí descubrimos un *método sistemático* para identificar problemas indecidibles: reducirlos a problemas que ya conocemos como indecidibles. La reducción $\leq_m$ es la herramienta básica de toda la teoría subsiguiente, y reaparecerá en forma de *reducciones polinomiales* cuando estudiemos complejidad computacional (clases P, NP, NP-completitud).

Fin del resumen teórico. Para profundizar, leer el Capítulo 5 completo de Sipser y revisar los ejercicios del Práctico 02.